Lab Manual – NoSQL DB

1. **Create an Employee collection consisting of Employee ID, Name, Age, Gender, Department and Address. Perform the following CRUD operations using MongoDB**
   a. **Insert details of the employees as documents.**
   b. **Retrieve the employee details whose age is less than or equal to 40 and display the same.**
   c. **Display the employees whose salary is less than 50k.**
   d. **Demonstrate Update operation.**
   e. **Remove the details of the employees belongs to any particular department.**

```
use employeeDB;

db.createCollection("Employee")

switched to db employeeDB;

db.Employee.insertMany([

 { EmpID: 101, Name: "Ravi Kumar", Age: 35, Gender: "Male", Department: "IT", Salary: 60000, Address: "Mysore" },

 { EmpID: 102, Name: "Asha R", Age: 42, Gender: "Female", Department: "HR", Salary: 48000, Address: "Bangalore" },

 { EmpID: 103, Name: "Kiran", Age: 29, Gender: "Male", Department: "Finance", Salary: 52000, Address: "Mangalore" },

 { EmpID: 104, Name: "Priya", Age: 39, Gender: "Female", Department: "IT", Salary: 45000, Address: "Mysore" },

 { EmpID: 105, Name: "Vikram", Age: 41, Gender: "Male", Department: "Sales", Salary: 55000, Address: "Hubli" }

])

{

 acknowledged: true,

 insertedIds: {

  '0': ObjectId('68abd8d00420f989304d6990'),

  '1': ObjectId('68abd8d00420f989304d6991'),

  '2': ObjectId('68abd8d00420f989304d6992'),

  '3': ObjectId('68abd8d00420f989304d6993'),

  '4': ObjectId('68abd8d00420f989304d6994')
```

```
 }
}
```

db.Employee.find({ Age: { $lte: 40 } })

```
{
 _id: ObjectId('68abd8d00420f989304d6990'),
 EmpID: 101,
 Name: 'Ravi Kumar',
 Age: 35,
 Gender: 'Male',
 Department: 'IT',
 Salary: 60000,
 Address: 'Mysore'
}
{
 _id: ObjectId('68abd8d00420f989304d6992'),
 EmpID: 103,
 Name: 'Kiran',
 Age: 29,
 Gender: 'Male',
 Department: 'Finance',
 Salary: 52000,
 Address: 'Mangalore'
}
{
 _id: ObjectId('68abd8d00420f989304d6993'),
 EmpID: 104,
 Name: 'Priya',
 Age: 39,
 Gender: 'Female',
 Department: 'IT',
```

Salary: 45000,

Address: 'Mysore'

}

db.Employee.find({ Salary: { $lt: 50000 } })

{

_id: ObjectId('68abd8d00420f989304d6991'),

EmpID: 102,

Name: 'Asha R',

Age: 42,

Gender: 'Female',

Department: 'HR',

Salary: 48000,

Address: 'Bangalore'

}

{

_id: ObjectId('68abd8d00420f989304d6993'),

EmpID: 104,

Name: 'Priya',

Age: 39,

Gender: 'Female',

Department: 'IT',

Salary: 45000,

Address: 'Mysore'

}

db.Employee.updateOne(

{ Name: "Priya" },

{ $set: { Salary: 55000 } }

)

{

acknowledged: true,

```
 insertedId: null,
 matchedCount: 1,
 modifiedCount: 1,
 upsertedCount: 0
}
db.Employee.updateOne(
 { Name: "Kiran" },
 { $set: { Department: "Accounts" } }
)
{
 acknowledged: true,
 insertedId: null,
 matchedCount: 1,
 modifiedCount: 1,
 upsertedCount: 0
}
db.Employee.deleteMany({ Department: "HR" })
{
 acknowledged: true,
 deletedCount: 1
}
```

2. **Create a university database containing collections for students, courses, departments, enrollments, and faculty. Each collection will have schema validation rules (e.g., minimum age, valid dates).Insert sample data and perform aggregation queries involving $lookup, $match, $group, $project, $sort, $unwind, and $facet to produce complex reports.**
   a. **Switch to database**
   b. **Create collections with validation**
   c. **Insert sample data (10+ docs each)**
   d. **Aggregation queries**
      i. **List each student with department name**
      ii. **Show each student's enrolled courses with course names**
      iii. **Number of students per department**

iv. Faculty list with their courses and departments

v. Department summary - total students, total faculty, total courses

1. a. use university

b. // Departments

```
db.createCollection("departments", {
 validator: {
  $jsonSchema: {
   bsonType: "object",
   required: ["deptId","name","establishedYear"],
   properties: {
    deptId: { bsonType: "int" },
    name: { bsonType: "string", minLength: 2 },
    establishedYear: { bsonType: "int", minimum: 1950, maximum: 2100 }
   }
  }
 }
});

// Students
db.createCollection("students", {
 validator: {
  $jsonSchema: {
   bsonType: "object",
   required: ["studentId","name","age","gender","deptId","admissionDate"],
   properties: {
    studentId: { bsonType: "int" },
    name: { bsonType: "string", minLength: 3 },
    age: { bsonType: "int", minimum: 16, maximum: 120 },
    gender: { enum: ["Male","Female","Other"] },
    deptId: { bsonType: "int" },
    admissionDate: { bsonType: "date" },
    email: { bsonType: "string", pattern: "^\\S+@\\S+\\.\\S+$" }
   }
  }
 }
});

// Faculty
db.createCollection("faculty", {
 validator: {
  $jsonSchema: {
   bsonType: "object",
   required: ["facultyId","name","deptId","hireDate","title"],
   properties: {
```

```
      facultyId: { bsonType: "int" },
      name: { bsonType: "string", minLength: 3 },
      deptId: { bsonType: "int" },
      hireDate: { bsonType: "date" },
      title: { bsonType: "string" },
      email: { bsonType: "string", pattern: "^\\S+@\\S+\\.\\S+$" }
    }
   }
  }
});

// Courses
db.createCollection("courses", {
 validator: {
  $jsonSchema: {
   bsonType: "object",
   required: ["courseId","name","deptId","credits"],
   properties: {
    courseId: { bsonType: "int" },
    name: { bsonType: "string", minLength: 3 },
    deptId: { bsonType: "int" },
    credits: { bsonType: "int", minimum: 1, maximum: 6 },
    level: { enum: ["UG","PG"] }
   }
  }
 }
});

// Enrollments
db.createCollection("enrollments", {
 validator: {
  $jsonSchema: {
   bsonType: "object",
   required: ["enrollId","studentId","courseId","semester","enrolledOn"],
   properties: {
    enrollId: { bsonType: "int" },
    studentId: { bsonType: "int" },
    courseId: { bsonType: "int" },
    semester: { bsonType: "string", pattern: "^(Odd|Even) 20\\d{2}-\\d{2}$" },
    enrolledOn: { bsonType: "date" },
    grade: { bsonType: ["string","null"], pattern: "^(A|B|C|D|E|F)$" }
   }
  }
 }
});
```

c. // Departments (5)

```
db.departments.insertMany([
 {deptId: 10, name: "Computer Science & Engineering", establishedYear: 1998},
 {deptId: 20, name: "Information Science & Engineering", establishedYear: 2001},
 {deptId: 30, name: "Electronics & Communication", establishedYear: 1995},
 {deptId: 40, name: "Mechanical Engineering", establishedYear: 1985},
 {deptId: 50, name: "Artificial Intelligence & Machine Learning", establishedYear: 2020}
]);

// Students (10+)
db.students.insertMany([
 {studentId: 1001, name: "Ananya S", age: 19, gender: "Female", deptId: 50, admissionDate:
ISODate("2023-08-01"), email:"ananya@univ.edu"},
 {studentId: 1002, name: "Rahul K", age: 20, gender: "Male", deptId: 10, admissionDate:
ISODate("2022-08-01"), email:"rahulk@univ.edu"},
 {studentId: 1003, name: "Priya R", age: 21, gender: "Female", deptId: 20, admissionDate:
ISODate("2022-08-01"), email:"priya.r@univ.edu"},
 {studentId: 1004, name: "Vivek N", age: 22, gender: "Male", deptId: 10, admissionDate:
ISODate("2021-08-01"), email:"vivek@univ.edu"},
 {studentId: 1005, name: "Ishaan M", age: 20, gender: "Male", deptId: 30, admissionDate:
ISODate("2022-08-01"), email:"ishaan@univ.edu"},
 {studentId: 1006, name: "Meghana P", age: 19, gender: "Female", deptId: 50,
admissionDate: ISODate("2023-08-01"), email:"meghana@univ.edu"},
 {studentId: 1007, name: "Sahana V", age: 20, gender: "Female", deptId: 20, admissionDate:
ISODate("2022-08-01"), email:"sahana@univ.edu"},
 {studentId: 1008, name: "Karthik T", age: 21, gender: "Male", deptId: 30, admissionDate:
ISODate("2021-08-01"), email:"karthik@univ.edu"},
 {studentId: 1009, name: "Deepa H", age: 22, gender: "Female", deptId: 40, admissionDate:
ISODate("2021-08-01"), email:"deepa@univ.edu"},
 {studentId: 1010, name: "Manoj B", age: 19, gender: "Male", deptId: 10, admissionDate:
ISODate("2023-08-01"), email:"manoj@univ.edu"},
 {studentId: 1011, name: "Neha J", age: 20, gender: "Female", deptId: 50, admissionDate:
ISODate("2023-08-01"), email:"neha@univ.edu"}
]);

// Faculty (10+)
db.faculty.insertMany([
 {facultyId: 201, name:"Dr. Akshay Rao", deptId:10, hireDate:ISODate("2015-07-01"),
title:"Professor", email:"akshay@univ.edu"},
 {facultyId: 202, name:"Dr. Revathi S", deptId:50, hireDate:ISODate("2021-02-01"),
title:"Associate Professor", email:"revathi@univ.edu"},
 {facultyId: 203, name:"Prof. Manjunath", deptId:20, hireDate:ISODate("2018-01-10"),
title:"Assistant Professor", email:"manju@univ.edu"},
 {facultyId: 204, name:"Dr. Priyanka N", deptId:30, hireDate:ISODate("2016-06-15"),
title:"Professor", email:"priyanka@univ.edu"},
 {facultyId: 205, name:"Prof. Shweta G", deptId:40, hireDate:ISODate("2019-08-20"),
title:"Assistant Professor", email:"shweta@univ.edu"},
```

```
 {facultyId: 206, name:"Dr. Raghav", deptId:10, hireDate:ISODate("2012-12-01"),
title:"Professor", email:"raghav@univ.edu"},
 {facultyId: 207, name:"Prof. Harish", deptId:20, hireDate:ISODate("2020-03-01"),
title:"Assistant Professor", email:"harish@univ.edu"},
 {facultyId: 208, name:"Dr. Meera", deptId:50, hireDate:ISODate("2022-09-01"),
title:"Assistant Professor", email:"meera@univ.edu"},
 {facultyId: 209, name:"Prof. Sudeep", deptId:30, hireDate:ISODate("2017-07-01"),
title:"Assistant Professor", email:"sudeep@univ.edu"},
 {facultyId: 210, name:"Dr. Kavitha", deptId:40, hireDate:ISODate("2014-11-11"),
title:"Professor", email:"kavitha@univ.edu"},
 {facultyId: 211, name:"Prof. Ritesh", deptId:10, hireDate:ISODate("2023-01-10"),
title:"Assistant Professor", email:"ritesh@univ.edu"}
]);

// Courses (10+)
db.courses.insertMany([
 {courseId: 301, name:"Data Structures", deptId:10, credits:4, level:"UG"},
 {courseId: 302, name:"Algorithms", deptId:10, credits:4, level:"UG"},
 {courseId: 303, name:"Database Systems", deptId:20, credits:3, level:"UG"},
 {courseId: 304, name:"Operating Systems", deptId:10, credits:4, level:"UG"},
 {courseId: 305, name:"Digital Signal Processing", deptId:30, credits:4, level:"UG"},
 {courseId: 306, name:"Microprocessors", deptId:30, credits:3, level:"UG"},
 {courseId: 307, name:"Thermodynamics", deptId:40, credits:3, level:"UG"},
 {courseId: 308, name:"Fluid Mechanics", deptId:40, credits:4, level:"UG"},
 {courseId: 309, name:"Machine Learning", deptId:50, credits:4, level:"UG"},
 {courseId: 310, name:"Deep Learning", deptId:50, credits:4, level:"PG"},
 {courseId: 311, name:"Information Retrieval", deptId:20, credits:3, level:"UG"}
]);

// Enrollments (20+)
db.enrollments.insertMany([
 {enrollId: 4001, studentId:1001, courseId:309, semester:"Odd 2024-25",
enrolledOn:ISODate("2024-09-01"), grade:null},
 {enrollId: 4002, studentId:1001, courseId:301, semester:"Odd 2024-25",
enrolledOn:ISODate("2024-09-01"), grade:null},
 {enrollId: 4003, studentId:1002, courseId:301, semester:"Odd 2023-24",
enrolledOn:ISODate("2023-09-01"), grade:"A"},
 {enrollId: 4004, studentId:1002, courseId:302, semester:"Even 2023-24",
enrolledOn:ISODate("2024-02-01"), grade:"B"},
 {enrollId: 4005, studentId:1003, courseId:303, semester:"Odd 2023-24",
enrolledOn:ISODate("2023-09-01"), grade:"A"},
 {enrollId: 4006, studentId:1003, courseId:311, semester:"Even 2023-24",
enrolledOn:ISODate("2024-02-01"), grade:"A"},
 {enrollId: 4007, studentId:1004, courseId:304, semester:"Odd 2022-23",
enrolledOn:ISODate("2022-09-01"), grade:"B"},
```

```
 {enrollId: 4008, studentId:1004, courseId:302, semester:"Even 2022-23",
enrolledOn:ISODate("2023-02-01"), grade:"A"},
 {enrollId: 4009, studentId:1005, courseId:305, semester:"Odd 2023-24",
enrolledOn:ISODate("2023-09-01"), grade:"C"},
 {enrollId: 4010, studentId:1005, courseId:306, semester:"Even 2023-24",
enrolledOn:ISODate("2024-02-01"), grade:"B"},
 {enrollId: 4011, studentId:1006, courseId:309, semester:"Odd 2024-25",
enrolledOn:ISODate("2024-09-01"), grade:null},
 {enrollId: 4012, studentId:1006, courseId:310, semester:"Odd 2024-25",
enrolledOn:ISODate("2024-09-01"), grade:null},
 {enrollId: 4013, studentId:1007, courseId:303, semester:"Odd 2023-24",
enrolledOn:ISODate("2023-09-01"), grade:"B"},
 {enrollId: 4014, studentId:1007, courseId:311, semester:"Even 2023-24",
enrolledOn:ISODate("2024-02-01"), grade:"A"},
 {enrollId: 4015, studentId:1008, courseId:305, semester:"Odd 2022-23",
enrolledOn:ISODate("2022-09-01"), grade:"B"},
 {enrollId: 4016, studentId:1008, courseId:308, semester:"Even 2022-23",
enrolledOn:ISODate("2023-02-01"), grade:"B"},
 {enrollId: 4017, studentId:1009, courseId:307, semester:"Odd 2022-23",
enrolledOn:ISODate("2022-09-01"), grade:"A"},
 {enrollId: 4018, studentId:1009, courseId:308, semester:"Even 2022-23",
enrolledOn:ISODate("2023-02-01"), grade:"A"},
 {enrollId: 4019, studentId:1010, courseId:302, semester:"Odd 2024-25",
enrolledOn:ISODate("2024-09-01"), grade:null},
 {enrollId: 4020, studentId:1010, courseId:304, semester:"Odd 2024-25",
enrolledOn:ISODate("2024-09-01"), grade:null},
 {enrollId: 4021, studentId:1011, courseId:309, semester:"Odd 2024-25",
enrolledOn:ISODate("2024-09-01"), grade:null},
 {enrollId: 4022, studentId:1011, courseId:310, semester:"Odd 2024-25",
enrolledOn:ISODate("2024-09-01"), grade:null}
]);
d. i. db.students.aggregate([
 {$lookup:{
  from:"departments",
  localField:"deptId",
  foreignField:"deptId",
  as:"dept"
 }},
 {$unwind:"$dept"},
 {$project:{_id:0, studentId:1, name:1, dept:"$dept.name"}}
]);

ii. db.students.aggregate([
 {$lookup:{
  from:"enrollments",
  localField:"studentId",
```

```
    foreignField:"studentId",
    as:"enrs"
   }},
   {$unwind:"$enrs"},
   {$lookup:{
    from:"courses",
    localField:"enrs.courseId",
    foreignField:"courseId",
    as:"course"
   }},
   {$unwind:"$course"},
   {$project:{
    _id:0, studentId:1, name:1,
    courseId:"$course.courseId",
    courseName:"$course.name",
    semester:"$enrs.semester", grade:"$enrs.grade"
   }},
   {$sort:{studentId:1, courseId:1}}
]);
```

iii.
```
db.students.aggregate([
 {$group:{_id:"$deptId", count:{$sum:1}}},
 {$lookup:{
  from:"departments",
  localField:"_id",
  foreignField:"deptId",
  as:"dept"
 }},
 {$unwind:"$dept"},
 {$project:{_id:0, deptId:"$dept.deptId", department:"$dept.name", students:"$count"}},
 {$sort:{deptId:1}}
]);
```

iv.
```
db.faculty.aggregate([
 {$lookup:{
  from:"departments",
  localField:"deptId",
  foreignField:"deptId",
  as:"dept"
 }},
 {$unwind:"$dept"},
 {$lookup:{
  from:"courses",
  let:{d:"$deptId"},
  pipeline:[
   {$match:{$expr:{$eq:["$deptId","$$d"]}}}
```

```
    ],
    as:"courses"
  }},
  {$project:{
    _id:0,
    facultyId:1, name:1, title:1,
    department:"$dept.name",
    courses:"$courses.name"
  }},
  {$sort:{department:1, name:1}}
]);
```

V.
```
db.departments.aggregate([
  {
    $facet: {
      students: [
        {$lookup:{from:"students", localField:"deptId", foreignField:"deptId", as:"s"}},
        {$project:{deptId:1, name:1, studentCount:{$size:"$s"}}}
      ],
      faculty: [
        {$lookup:{from:"faculty", localField:"deptId", foreignField:"deptId", as:"f"}},
        {$project:{deptId:1, name:1, facultyCount:{$size:"$f"}}}
      ],
      courses: [
        {$lookup:{from:"courses", localField:"deptId", foreignField:"deptId", as:"c"}},
        {$project:{deptId:1, name:1, courseCount:{$size:"$c"}}}
      ]
    }
  },
  // Stitch the three facets by deptId
  {
    $project: {
      merged: {
        $map: {
          input: "$students",
          as: "s",
          in: {
            deptId: "$$s.deptId",
            department: "$$s.name",
            studentCount: "$$s.studentCount",
            facultyCount: {
              $let: {
                vars: { f: { $arrayElemAt: [
                  { $filter: { input: "$faculty", as:"ff", cond: {$eq:["$$ff.deptId","$$s.deptId"]} } }, 0 ] }
},
                in: "$$f.facultyCount"
```

```
            }
           },
           courseCount: {
            $let: {
             vars: { c: { $arrayElemAt: [
              { $filter: { input: "$courses", as:"cc", cond: {$eq:["$$cc.deptId","$$s.deptId"]} } }, 0
] } },
             in: "$$c.courseCount"
            }
           }
          }
         }
        }
       },
       {$unwind:"$merged"},
       {$replaceRoot:{newRoot:"$merged"}},
       {$sort:{deptId:1}}
      ]);
```

Additional:

```
db.students.createIndex({deptId:1, studentId:1});
db.faculty.createIndex({deptId:1});
db.courses.createIndex({deptId:1, courseId:1});
db.enrollments.createIndex({studentId:1, courseId:1});
db.departments.createIndex({deptId:1}, {unique:true});
```

3. **Create a pharmacy database containing customers, medicines, suppliers, orders, and pharmacists. Schema validation will enforce constraints like: age must be ≥ 18 for customers Expiry Date must be a valid ISO date in medicines Stock must be ≥ 0 Insert ≥10 documents into each collection and perform multi-collection aggregations.**
   a. **Switch to database**
   b. **Create collections with validation**
   c. **Insert sample data (10 docs each)**
   d. **Aggregation queries**
      i. **List all orders with customer name and medicine name**
      ii. **Total sales per medicine**
      iii. **Supplier-wise revenue**
      iv. **Medicines below stock threshold (e.g., < 70 units)**
      v. **Customer order summary**

a. use pharmacyDB

b. Collections:

```
db.createCollection("customers", {
```

```
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["customer_id","name","age","phone"],
      properties: {
        customer_id: { bsonType: "int" },
        name: { bsonType: "string" },
        age: { bsonType: "int", minimum: 18 },
        phone: { bsonType: "string" }
      }
    }
  }
})

db.createCollection("suppliers", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["supplier_id","name","contact"],
      properties: {
        supplier_id: { bsonType: "int" },
        name: { bsonType: "string" },
        contact: { bsonType: "string" }
      }
    }
  }
})

db.createCollection("pharmacists", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["pharmacist_id","name","shift"],
      properties: {
        pharmacist_id: { bsonType: "int" },
        name: { bsonType: "string" },
        shift: { enum: ["Morning","Evening","Night"] }
      }
    }
  }
})
```

```
db.createCollection("medicines", {
 validator: {
  $jsonSchema: {
   bsonType: "object",
   required: ["medicine_id","name","price","stock","expiry"],
   properties: {
    medicine_id: { bsonType: "int" },
    name:       { bsonType: "string" },
    price:      { bsonType: ["double","int"] },
    stock:      { bsonType: "int", minimum: 0 },
    expiry:     { bsonType: "date" }
   }
  }
 }
})

db.createCollection("orders", {
 validator: {
  $jsonSchema: {
   bsonType: "object",
   required: [
    "order_id",
    "customer_id",
    "medicine_id",
    "quantity",
    "pharmacist_id",
    "total"
   ],
   properties: {
    order_id:     { bsonType: "int" },
    customer_id:  { bsonType: "int" },
    medicine_id:  { bsonType: "int" },
    pharmacist_id: { bsonType: "int" },
    quantity:     { bsonType: "int", minimum: 1 },
    total:        { bsonType: ["double", "int"], minimum: 0 }
   }
  }
 }
})
```

c. Insertions:

```
db.orders.insertMany([

{order_id:NumberInt(1),customer_id:NumberInt(1),medicine_id:NumberInt(1),quantity:NumberInt(5),pharmacist_id:NumberInt(1),total:100.0},

{order_id:NumberInt(2),customer_id:NumberInt(2),medicine_id:NumberInt(2),quantity:NumberInt(2),pharmacist_id:NumberInt(2),total:100.0},

{order_id:NumberInt(3),customer_id:NumberInt(3),medicine_id:NumberInt(3),quantity:NumberInt(3),pharmacist_id:NumberInt(3),total:90.0},

{order_id:NumberInt(4),customer_id:NumberInt(4),medicine_id:NumberInt(4),quantity:NumberInt(4),pharmacist_id:NumberInt(1),total:60.0},

{order_id:NumberInt(5),customer_id:NumberInt(5),medicine_id:NumberInt(5),quantity:NumberInt(2),pharmacist_id:NumberInt(2),total:180.0},

{order_id:NumberInt(6),customer_id:NumberInt(6),medicine_id:NumberInt(6),quantity:NumberInt(6),pharmacist_id:NumberInt(3),total:150.0},

{order_id:NumberInt(7),customer_id:NumberInt(7),medicine_id:NumberInt(7),quantity:NumberInt(1),pharmacist_id:NumberInt(1),total:300.0},

{order_id:NumberInt(8),customer_id:NumberInt(8),medicine_id:NumberInt(8),quantity:NumberInt(8),pharmacist_id:NumberInt(2),total:144.0},

{order_id:NumberInt(9),customer_id:NumberInt(9),medicine_id:NumberInt(9),quantity:NumberInt(2),pharmacist_id:NumberInt(3),total:80.0},

{order_id:NumberInt(10),customer_id:NumberInt(10),medicine_id:NumberInt(10),quantity:NumberInt(1),pharmacist_id:NumberInt(1),total:60.0}
])


db.medicines.insertMany([

{medicine_id:NumberInt(1),name:"Paracetamol",price:20.0,stock:NumberInt(120),expiry:ISODate("2025-12-31")},

{medicine_id:NumberInt(2),name:"Amoxicillin",price:50.0,stock:NumberInt(80),expiry:ISODate("2025-06-30")},
```

```
{medicine_id:NumberInt(3),name:"Ibuprofen",price:30.0,stock:NumberInt(60
),expiry:ISODate("2024-09-30")},

{medicine_id:NumberInt(4),name:"Cetirizine",price:15.0,stock:NumberInt(10
0),expiry:ISODate("2025-03-31")},
 {medicine_id:NumberInt(5),name:"Cough
Syrup",price:90.0,stock:NumberInt(45),expiry:ISODate("2024-11-30")},
 {medicine_id:NumberInt(6),name:"Vitamin
C",price:25.0,stock:NumberInt(200),expiry:ISODate("2026-01-01")},

{medicine_id:NumberInt(7),name:"Insulin",price:300.0,stock:NumberInt(35),e
xpiry:ISODate("2024-07-15")},

{medicine_id:NumberInt(8),name:"Antacid",price:18.0,stock:NumberInt(150),
expiry:ISODate("2026-05-01")},
 {medicine_id:NumberInt(9),name:"Pain
Balm",price:40.0,stock:NumberInt(95),expiry:ISODate("2025-08-20")},
 {medicine_id:NumberInt(10),name:"Eye
Drops",price:60.0,stock:NumberInt(50),expiry:ISODate("2024-10-05")}
])

db.customers.insertMany([
 {customer_id:1,name:"Alice",age:25,phone:"9876543210"},
 {customer_id:2,name:"Bob",age:30,phone:"9876500011"},
 {customer_id:3,name:"Charlie",age:28,phone:"9876500022"},
 {customer_id:4,name:"David",age:35,phone:"9876500033"},
 {customer_id:5,name:"Eva",age:40,phone:"9876500044"},
 {customer_id:6,name:"Frank",age:22,phone:"9876500055"},
 {customer_id:7,name:"Grace",age:26,phone:"9876500066"},
 {customer_id:8,name:"Helen",age:31,phone:"9876500077"},
 {customer_id:9,name:"Ivan",age:38,phone:"9876500088"},
 {customer_id:10,name:"Jack",age:29,phone:"9876500099"}
])

db.suppliers.insertMany([
 {supplier_id:1,name:"MediSupply Co.",contact:"987600001"},
 {supplier_id:2,name:"HealthFirst Ltd.",contact:"987600002"},
 {supplier_id:3,name:"CareWell Pharma",contact:"987600003"}
])

db.pharmacists.insertMany([
```

```
{pharmacist_id:1,name:"Dr. Smith",shift:"Morning"},
{pharmacist_id:2,name:"Dr. Rose",shift:"Evening"},
{pharmacist_id:3,name:"Dr. John",shift:"Night"}
])
```

e. Aggregations:

```
db.orders.aggregate([
 { $lookup: {
    from: "customers",
    localField: "customer_id",
    foreignField: "customer_id",
    as: "customer"
 }},
 { $lookup: {
    from: "medicines",
    localField: "medicine_id",
    foreignField: "medicine_id",
    as: "medicine"
 }},
 { $project: {
    order_id: 1,
    customer_name: { $arrayElemAt: ["$customer.name",0] },
    medicine_name: { $arrayElemAt: ["$medicine.name",0] },
    quantity:1,
    total:1
 }}
])

db.orders.aggregate([
 { $group: { _id: "$medicine_id", totalSales: { $sum: "$total" } } },
 { $lookup: {
    from: "medicines",
    localField: "_id",
    foreignField: "medicine_id",
    as: "medicine"
 }},
 { $project: { medicine_name: { $arrayElemAt: ["$medicine.name",0] }, totalSales: 1 } }
])

// Add supplier_id to medicines before running this.
db.medicines.updateMany({},[{$set:{supplier_id:1}}]) // Example
```

```
db.orders.aggregate([
 { $lookup: {
    from: "medicines",
    localField: "medicine_id",
    foreignField: "medicine_id",
    as: "med"
 }},
 { $unwind: "$med" },
 { $group: { _id: "$med.supplier_id", revenue: { $sum: "$total" } } },
 { $lookup: {
    from: "suppliers",
    localField: "_id",
    foreignField: "supplier_id",
    as: "supplier"
 }},
 { $project: { supplier_name: { $arrayElemAt: ["$supplier.name",0] }, revenue: 1 } }
])

db.medicines.find({ stock: { $lt: 70 } }, { name:1, stock:1 })

db.orders.aggregate([
 { $group: {
    _id: "$customer_id",
    totalOrders: { $sum: 1 },
    totalAmount: { $sum: "$total" }
 }},
 { $lookup: {
    from: "customers",
    localField: "_id",
    foreignField: "customer_id",
    as: "customer"
 }},
 { $project: {
    customer_name: { $arrayElemAt: ["$customer.name",0] },
    totalOrders:1,
    totalAmount:1
 }}
])
```

4. **Create patient database with 5 collections: patients; doctors; medicines; prescriptions; sales Schema validation will ensure fields like age are within a valid range, dates follow ISODate format, and IDs are unique. Then create different indexes and run queries to show their use.**
   a. **Create Database and Collections with Validation**
   b. **Insert Sample Data (10 Documents per Collection)**

```
// Switch to database
use("patient_db");

// Create patients collection with validation
db.createCollection("patients", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["patient_id", "name", "age", "gender", "contact", "created_at"],
      properties: {
        patient_id: { bsonType: "string" },
        name: { bsonType: "string" },
        age: { bsonType: "int", minimum: 0, maximum: 120 },
        gender: { enum: ["Male", "Female", "Other"] },
        contact: { bsonType: "string" },
        address: { bsonType: "string" },
        created_at: { bsonType: "date" }
      }
    }
  }
});

// Create doctors collection
db.createCollection("doctors", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["doctor_id", "name", "specialty", "contact", "created_at"],
      properties: {
        doctor_id: { bsonType: "string" },
        name: { bsonType: "string" },
        specialty: { bsonType: "string" },
        contact: { bsonType: "string" },
        created_at: { bsonType: "date" }
      }
    }
```

```
  }
});

// Create medicines collection
db.runCommand({
  collMod: "medicines",
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["medicine_id","name","manufacturer","expiry_date","price"],
      properties: {
        medicine_id: { bsonType: "string" },
        name: { bsonType: "string" },
        manufacturer: { bsonType: "string" },
        expiry_date: { bsonType: "date" },
        // accept int, double, decimal for price
        price: { bsonType: ["double","int","decimal"], minimum: 0 }
      }
    }
  },
  validationLevel: "moderate"
});

// Create prescriptions collection
db.createCollection("prescriptions", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["prescription_id", "patient_id", "doctor_id", "medicines", "date_issued"],
      properties: {
        prescription_id: { bsonType: "string" },
        patient_id: { bsonType: "string" },
        doctor_id: { bsonType: "string" },
        medicines: {
          bsonType: "array",
          items: {
            bsonType: "object",
            required: ["medicine_id", "dosage", "duration"],
            properties: {
              medicine_id: { bsonType: "string" },
              dosage: { bsonType: "string" },
              duration: { bsonType: "string" }
            }
          }
```

```
          },
          date_issued: { bsonType: "date" },
          notes: { bsonType: "string" }
        }
      }
    }
  });

  //  Create sales collection
db.runCommand({
 collMod: "sales",
 validator: {
  $jsonSchema: {
   bsonType: "object",
   required: ["sale_id","medicine_id","quantity","total_price","sale_date"],
   properties: {
    sale_id: { bsonType: "string" },
    medicine_id: { bsonType: "string" },
    quantity: { bsonType: "int", minimum: 1 },
    total_price: { bsonType: ["double","int","decimal"], minimum: 0 },
    sale_date: { bsonType: "date" },
    customer_name: { bsonType: "string" }
   }
  }
 },
 validationLevel: "moderate"
});

// -- patients (10)
db.patients.insertMany([
 { patient_id: "P001", name: "Asha Rao", age: 25, gender: "Female", contact: "987650001",
address: "Bengaluru", created_at: ISODate("2024-01-10") },
 { patient_id: "P002", name: "Ravi Kumar", age: 40, gender: "Male", contact: "987650002",
address: "Mysuru", created_at: ISODate("2024-03-05") },
 { patient_id: "P003", name: "Sita Devi", age: 33, gender: "Female", contact: "987650003",
address: "Hubli", created_at: ISODate("2024-02-20") },
 { patient_id: "P004", name: "Arjun N", age: 60, gender: "Male", contact: "987650004",
address: "Mangalore", created_at: ISODate("2023-11-15") },
 { patient_id: "P005", name: "Leela", age: 29, gender: "Female", contact: "987650005",
address: "Belgaum", created_at: ISODate("2023-12-01") },
 { patient_id: "P006", name: "Manish", age: 50, gender: "Male", contact: "987650006",
address: "Bengaluru", created_at: ISODate("2024-04-02") },
 { patient_id: "P007", name: "Priya", age: 20, gender: "Female", contact: "987650007",
address: "Tumkur", created_at: ISODate("2024-05-12") },
```

```
{ patient_id: "P008", name: "Karan", age: 70, gender: "Male", contact: "987650008", address: "Dharwad", created_at: ISODate("2023-09-30") },
{ patient_id: "P009", name: "Nila", age: 35, gender: "Female", contact: "987650009", address: "Udupi", created_at: ISODate("2024-06-18") },
{ patient_id: "P010", name: "Suresh", age: 45, gender: "Male", contact: "987650010", address: "Gulbarga", created_at: ISODate("2024-07-01") }
])

// -- doctors (10)
db.doctors.insertMany([
{ doctor_id: "D001", name: "Dr. Meera", specialty: "Cardiology", contact: "990001001", created_at: ISODate("2022-02-10") },
{ doctor_id: "D002", name: "Dr. Raj", specialty: "General", contact: "990001002", created_at: ISODate("2023-01-20") },
{ doctor_id: "D003", name: "Dr. Nisha", specialty: "Dermatology", contact: "990001003", created_at: ISODate("2021-10-12") },
{ doctor_id: "D004", name: "Dr. Bhat", specialty: "Orthopedics", contact: "990001004", created_at: ISODate("2023-05-05") },
{ doctor_id: "D005", name: "Dr. Patel", specialty: "Pediatrics", contact: "990001005", created_at: ISODate("2024-02-02") },
{ doctor_id: "D006", name: "Dr. Sunil", specialty: "Neurology", contact: "990001006", created_at: ISODate("2022-09-09") },
{ doctor_id: "D007", name: "Dr. Anu", specialty: "Gynecology", contact: "990001007", created_at: ISODate("2024-06-01") },
{ doctor_id: "D008", name: "Dr. Mehta", specialty: "ENT", contact: "990001008", created_at: ISODate("2023-07-07") },
{ doctor_id: "D009", name: "Dr. Roy", specialty: "General", contact: "990001009", created_at: ISODate("2024-03-03") },
{ doctor_id: "D010", name: "Dr. Iyer", specialty: "Cardiology", contact: "990001010", created_at: ISODate("2021-11-11") }
])

// -- medicines (10)
db.medicines.insertMany([
{ medicine_id: "M001", name: "Paracetamol", manufacturer: "PharmaA", expiry_date: ISODate("2026-12-31"), price: 10.0 },
{ medicine_id: "M002", name: "Amoxicillin", manufacturer: "PharmaB", expiry_date: ISODate("2025-06-30"), price: 25.0 },
{ medicine_id: "M003", name: "Cetirizine", manufacturer: "PharmaA", expiry_date: ISODate("2027-02-28"), price: 12.0 },
{ medicine_id: "M004", name: "Atorvastatin", manufacturer: "PharmaC", expiry_date: ISODate("2026-08-15"), price: 50.0 },
{ medicine_id: "M005", name: "Metformin", manufacturer: "PharmaD", expiry_date: ISODate("2025-11-11"), price: 40.0 },
```

```
{ medicine_id: "M006", name: "Ibuprofen", manufacturer: "PharmaA", expiry_date: ISODate("2026-05-05"), price: 15.0 },
{ medicine_id: "M007", name: "Omeprazole", manufacturer: "PharmaB", expiry_date: ISODate("2027-09-09"), price: 30.0 },
{ medicine_id: "M008", name: "Aspirin", manufacturer: "PharmaE", expiry_date: ISODate("2025-03-03"), price: 8.0 },
{ medicine_id: "M009", name: "Insulin", manufacturer: "PharmaF", expiry_date: ISODate("2025-07-07"), price: 200.0 },
{ medicine_id: "M010", name: "VitaminC", manufacturer: "PharmaA", expiry_date: ISODate("2028-01-01"), price: 5.0 }
])

// -- prescriptions (10)
db.prescriptions.insertMany([
{ prescription_id: "PR001", patient_id: "P001", doctor_id: "D002", medicines: [{medicine_id:"M001",dosage:"500mg",duration:"5 days"}], date_issued: ISODate("2024-01-11"), notes: "Fever" },
{ prescription_id: "PR002", patient_id: "P002", doctor_id: "D001", medicines: [{medicine_id:"M004",dosage:"10mg",duration:"30 days"}], date_issued: ISODate("2024-03-06") },
{ prescription_id: "PR003", patient_id: "P003", doctor_id: "D003", medicines: [{medicine_id:"M003",dosage:"10mg",duration:"7 days"}], date_issued: ISODate("2024-02-22") },
{ prescription_id: "PR004", patient_id: "P004", doctor_id: "D004", medicines: [{medicine_id:"M006",dosage:"200mg",duration:"3 days"}], date_issued: ISODate("2023-11-16") },
{ prescription_id: "PR005", patient_id: "P005", doctor_id: "D005", medicines: [{medicine_id:"M007",dosage:"20mg",duration:"14 days"}], date_issued: ISODate("2023-12-02") },
{ prescription_id: "PR006", patient_id: "P006", doctor_id: "D006", medicines: [{medicine_id:"M009",dosage:"variable",duration:"ongoing"}], date_issued: ISODate("2024-04-03") },
{ prescription_id: "PR007", patient_id: "P007", doctor_id: "D007", medicines: [{medicine_id:"M010",dosage:"1 tab",duration:"10 days"}], date_issued: ISODate("2024-05-13") },
{ prescription_id: "PR008", patient_id: "P008", doctor_id: "D008", medicines: [{medicine_id:"M002",dosage:"500mg",duration:"7 days"}], date_issued: ISODate("2023-10-01") },
{ prescription_id: "PR009", patient_id: "P009", doctor_id: "D009", medicines: [{medicine_id:"M005",dosage:"500mg",duration:"90 days"}], date_issued: ISODate("2024-06-19") },
{ prescription_id: "PR010", patient_id: "P010", doctor_id: "D010", medicines: [{medicine_id:"M008",dosage:"75mg",duration:"7 days"}], date_issued: ISODate("2024-07-02") }
```

```
])

// -- sales (10)
db.sales.insertMany([
 { sale_id: "S001", medicine_id: "M001", quantity: 2, total_price: 20.0, sale_date: ISODate("2024-01-11"), customer_name: "Asha Rao" },
 { sale_id: "S002", medicine_id: "M004", quantity: 1, total_price: 50.0, sale_date: ISODate("2024-03-06"), customer_name: "Ravi Kumar" },
 { sale_id: "S003", medicine_id: "M003", quantity: 3, total_price: 36.0, sale_date: ISODate("2024-02-22"), customer_name: "Sita Devi" },
 { sale_id: "S004", medicine_id: "M006", quantity: 1, total_price: 15.0, sale_date: ISODate("2023-11-16"), customer_name: "Arjun N" },
 { sale_id: "S005", medicine_id: "M007", quantity: 2, total_price: 60.0, sale_date: ISODate("2023-12-02"), customer_name: "Leela" },
 { sale_id: "S006", medicine_id: "M009", quantity: 1, total_price: 200.0, sale_date: ISODate("2024-04-03"), customer_name: "Manish" },
 { sale_id: "S007", medicine_id: "M010", quantity: 5, total_price: 25.0, sale_date: ISODate("2024-05-13"), customer_name: "Priya" },
 { sale_id: "S008", medicine_id: "M002", quantity: 2, total_price: 50.0, sale_date: ISODate("2023-10-01"), customer_name: "Karan" },
 { sale_id: "S009", medicine_id: "M005", quantity: 1, total_price: 40.0, sale_date: ISODate("2024-06-19"), customer_name: "Nila" },
 { sale_id: "S010", medicine_id: "M008", quantity: 4, total_price: 32.0, sale_date: ISODate("2024-07-02"), customer_name: "Suresh" }
])
```

c. Creating Indexes

i. **Single Field Index**
ii. **Compound Index**
iii. **Text Index**
iv. **Partial Index**
v. **Unique Index**

```
// Single Field Index
db.patients.createIndex({ patient_id: 1 });

// Compound Index
db.prescriptions.createIndex({ patient_id: 1, date_issued: -1 });

// Text Index
db.medicines.createIndex({ name: "text", manufacturer: "text" });

// Partial Index (only where total_price > 50)
db.sales.createIndex(
```

```
  { total_price: 1 },
  { partialFilterExpression: { total_price: { $gt: 50 } } }
);

// Unique Index
db.doctors.createIndex({ doctor_id: 1 }, { unique: true });
```

d. Example Queries Using Indexes

```
// Query 1: Single-field index
db.patients.find({ patient_id: "P001" });

// Query 2: Compound index
db.prescriptions.find({ patient_id: "P001" }).sort({ date_issued: -1 });

// Query 3: Text index
db.medicines.find({ $text: { $search: "Paracetamol PharmaA" } });

// Query 4: Partial index
db.sales.find({ total_price: { $gt: 50 } });
```

e. Index Verification

```
// Show all indexes
db.patients.getIndexes();
db.prescriptions.getIndexes();
db.sales.getIndexes();

// Check explain plan (to confirm index usage)
db.prescriptions.find({ patient_id: "P001" }).sort({ date_issued: -1
}).explain("executionStats");
```

f. Performance Comparison Example

i. **Without Index**
ii. **With Index**

To see difference:

1. Run a query before creating index (time manually).
2. Then create the index and re-run the same query.

Or, duplicate the collection:

```
db.prescriptions.aggregate([{ $match: {} }, { $out: "prescriptions_copy" }]);
db.prescriptions_copy.dropIndexes();
```

Then compare query time on prescriptions (with index) vs prescriptions_copy (without index).

5. **Create a Library with 4 collections (books, members, borrowRecords, authors) and validation rules insert least 10 documents. In this exercise create indexes examples (single-field, compound, text, unique) and show performance test with explain ().**
   a. **Create & Use Database and Collections with Validation**
   b. **Insert Sample Documents (10 each)**
   c. **Index Creation**
      i. **Single-field index**
      ii. **Compound index**
      iii. **Text index**
      iv. **Unique index**
   d. **Performance Test with explain()**
      i. **Without Index**
      ii. **With Index**
   e. **Library Database — Additional Indexed Queries**
      i. **Find books by a specific author quickly**
      ii. **Search books by title keyword (text index)**
      iii. **Find overdue books efficiently**
      iv. **Get borrowing history of a specific member, sorted by date**

```
// ============ 0. Setup: Switch DB and clean (safe for demo) ============
use("library_db");

// WARNING: For an existing DB, drop lines below carefully in a lab/demo only.
db.books.drop();
db.members.drop();
db.authors.drop();
db.borrowRecords.drop();

// ============ 1. Create collections with validation ============

// books collection: book_id unique string, title string, authors array of author_id refs, isbn, publishedDate (date), pages (int), description
db.createCollection("books", {
 validator: {
  $jsonSchema: {
   bsonType: "object",
   required: ["book_id","title","authors","isbn","publishedDate","pages"],
   properties: {
    book_id: { bsonType: "string" },
    title: { bsonType: "string" },
    authors: {
     bsonType: "array",
     items: { bsonType: "string" }
```

```
      },
      isbn: { bsonType: "string" },
      publishedDate: { bsonType: "date" },
      pages: { bsonType: ["int","double","decimal"], minimum: 1 },
      description: { bsonType: "string" },
      categories: {
        bsonType: "array",
        items: { bsonType: "string" }
      }
    }
  }
 }
});

// members collection: member_id unique string, name, email, joinedDate
db.createCollection("members", {
 validator: {
  $jsonSchema: {
    bsonType: "object",
    required: ["member_id","name","email","joinedDate"],
    properties: {
      member_id: { bsonType: "string" },
      name: { bsonType: "string" },
      email: { bsonType: "string" },
      joinedDate: { bsonType: "date" }
    }
  }
 }
});

// authors collection: author_id unique string, name, bio
db.createCollection("authors", {
 validator: {
  $jsonSchema: {
    bsonType: "object",
    required: ["author_id","name"],
    properties: {
      author_id: { bsonType: "string" },
      name: { bsonType: "string" },
      bio: { bsonType: "string" }
    }
  }
 }
});
```

```
// borrowRecords collection: record_id unique, book_id, member_id, borrowDate, dueDate, returned (bool), returnDate(optional)
db.createCollection("borrowRecords", {
 validator: {
  $jsonSchema: {
   bsonType: "object",
   required: ["record_id","book_id","member_id","borrowDate","dueDate","returned"],
   properties: {
    record_id: { bsonType: "string" },
    book_id: { bsonType: "string" },
    member_id: { bsonType: "string" },
    borrowDate: { bsonType: "date" },
    dueDate: { bsonType: "date" },
    returned: { bsonType: "bool" },
    returnDate: { bsonType: "date" }
   }
  }
 }
});

// ============ 2. Insert sample documents (10 each) ============

// -- authors (10)
db.authors.insertMany([
 { author_id: "A001", name: "R. K. Narayan", bio: "Indian writer." },
 { author_id: "A002", name: "Arundhati Roy", bio: "Author & activist." },
 { author_id: "A003", name: "Ruskin Bond", bio: "Short story author." },
 { author_id: "A004", name: "Amrita Pritam", bio: "Poet & novelist." },
 { author_id: "A005", name: "Chetan Bhagat", bio: "Contemporary novelist." },
 { author_id: "A006", name: "A. S. Byatt", bio: "English novelist." },
 { author_id: "A007", name: "J. K. Rowling", bio: "Fantasy novelist." },
 { author_id: "A008", name: "George Orwell", bio: "Political novelist." },
 { author_id: "A009", name: "Jane Austen", bio: "Classic novelist." },
 { author_id: "A010", name: "Mark Twain", bio: "American author." }
]);

// -- books (10)
db.books.insertMany([
 { book_id: "B001", title: "Malgudi Days", authors: ["A001"], isbn: "ISBN-0001", publishedDate: ISODate("1943-01-01"), pages: 250, description: "Collection of short stories", categories: ["Fiction", "Short Stories"] },
 { book_id: "B002", title: "The God of Small Things", authors: ["A002"], isbn: "ISBN-0002", publishedDate: ISODate("1997-06-01"), pages: 340, description: "Novel about family", categories: ["Fiction"] },
```

```
{ book_id: "B003", title: "The Room on the Roof", authors: ["A003"], isbn: "ISBN-0003", publishedDate: ISODate("1956-01-01"), pages: 220, description: "Coming-of-age novel", categories: ["Fiction"] },
{ book_id: "B004", title: "Pinjar", authors: ["A004"], isbn: "ISBN-0004", publishedDate: ISODate("1950-01-01"), pages: 300, description: "Partition novel", categories: ["Historical"] },
{ book_id: "B005", title: "Five Point Someone", authors: ["A005"], isbn: "ISBN-0005", publishedDate: ISODate("2004-01-01"), pages: 280, description: "Campus story", categories: ["Fiction", "Young Adult"] },
{ book_id: "B006", title: "Possession", authors: ["A006"], isbn: "ISBN-0006", publishedDate: ISODate("1990-01-01"), pages: 480, description: "Literary novel", categories: ["Fiction"] },
{ book_id: "B007", title: "Harry Potter and the Philosopher's Stone", authors: ["A007"], isbn: "ISBN-0007", publishedDate: ISODate("1997-06-26"), pages: 223, description: "Fantasy novel", categories: ["Fantasy", "Young Adult"] },
{ book_id: "B008", title: "1984", authors: ["A008"], isbn: "ISBN-0008", publishedDate: ISODate("1949-06-08"), pages: 328, description: "Dystopian novel", categories: ["Fiction", "Political"] },
{ book_id: "B009", title: "Pride and Prejudice", authors: ["A009"], isbn: "ISBN-0009", publishedDate: ISODate("1813-01-28"), pages: 279, description: "Classic romance", categories: ["Classic"] },
{ book_id: "B010", title: "Adventures of Huckleberry Finn", authors: ["A010"], isbn: "ISBN-0010", publishedDate: ISODate("1884-12-10"), pages: 366, description: "American classic", categories: ["Classic"] }
]);

// -- members (10)
db.members.insertMany([
{ member_id: "M001", name: "Asha Rao", email: "asha@example.com", joinedDate: ISODate("2022-01-10") },
{ member_id: "M002", name: "Ravi Kumar", email: "ravi@example.com", joinedDate: ISODate("2021-03-05") },
{ member_id: "M003", name: "Sita Devi", email: "sita@example.com", joinedDate: ISODate("2023-02-20") },
{ member_id: "M004", name: "Arjun N", email: "arjun@example.com", joinedDate: ISODate("2020-11-15") },
{ member_id: "M005", name: "Leela", email: "leela@example.com", joinedDate: ISODate("2021-12-01") },
{ member_id: "M006", name: "Manish", email: "manish@example.com", joinedDate: ISODate("2022-04-02") },
{ member_id: "M007", name: "Priya", email: "priya@example.com", joinedDate: ISODate("2024-05-12") },
{ member_id: "M008", name: "Karan", email: "karan@example.com", joinedDate: ISODate("2020-09-30") },
{ member_id: "M009", name: "Nila", email: "nila@example.com", joinedDate: ISODate("2023-06-18") },
{ member_id: "M010", name: "Suresh", email: "suresh@example.com", joinedDate: ISODate("2021-07-01") }
]);
```

```
// -- borrowRecords (10)
db.borrowRecords.insertMany([
 { record_id: "R001", book_id: "B001", member_id: "M001", borrowDate: ISODate("2024-08-01"), dueDate: ISODate("2024-08-15"), returned: false },
 { record_id: "R002", book_id: "B002", member_id: "M002", borrowDate: ISODate("2024-07-10"), dueDate: ISODate("2024-07-24"), returned: true, returnDate: ISODate("2024-07-20") },
 { record_id: "R003", book_id: "B003", member_id: "M003", borrowDate: ISODate("2024-07-15"), dueDate: ISODate("2024-07-29"), returned: true, returnDate: ISODate("2024-07-28") },
 { record_id: "R004", book_id: "B004", member_id: "M004", borrowDate: ISODate("2024-06-20"), dueDate: ISODate("2024-07-04"), returned: true, returnDate: ISODate("2024-06-30") },
 { record_id: "R005", book_id: "B005", member_id: "M005", borrowDate: ISODate("2024-07-30"), dueDate: ISODate("2024-08-13"), returned: false },
 { record_id: "R006", book_id: "B006", member_id: "M006", borrowDate: ISODate("2024-06-01"), dueDate: ISODate("2024-06-15"), returned: true, returnDate: ISODate("2024-06-10") },
 { record_id: "R007", book_id: "B007", member_id: "M007", borrowDate: ISODate("2024-08-05"), dueDate: ISODate("2024-08-19"), returned: false },
 { record_id: "R008", book_id: "B008", member_id: "M008", borrowDate: ISODate("2024-07-01"), dueDate: ISODate("2024-07-15"), returned: true, returnDate: ISODate("2024-07-14") },
 { record_id: "R009", book_id: "B009", member_id: "M009", borrowDate: ISODate("2024-07-25"), dueDate: ISODate("2024-08-08"), returned: false },
 { record_id: "R010", book_id: "B010", member_id: "M010", borrowDate: ISODate("2024-06-11"), dueDate: ISODate("2024-06-25"), returned: true, returnDate: ISODate("2024-06-20") }
]);

// ============ 3. Index creation ============

// i) Single-field index on books.book_id (and on members.member_id) - helps fast lookup
db.books.createIndex({ book_id: 1 }, { name: "idx_books_book_id" });
db.members.createIndex({ member_id: 1 }, { name: "idx_members_member_id" });

// ii) Compound index: borrowRecords on (member_id, borrowDate desc) - fast borrowing history per member sorted by date
db.borrowRecords.createIndex({ member_id: 1, borrowDate: -1 }, { name: "idx_borrow_member_borrowDate" });

// iii) Text index: search on books.title and books.description
db.books.createIndex({ title: "text", description: "text" }, { name: "idx_books_text" });

// iv) Unique indexes: ensure unique IDs (create unique on id fields)
db.books.createIndex({ book_id: 1 }, { unique: true, name: "uq_books_book_id" });          // may be redundant with previous but ensures uniqueness
db.members.createIndex({ member_id: 1 }, { unique: true, name: "uq_members_member_id" });
db.authors.createIndex({ author_id: 1 }, { unique: true, name: "uq_authors_author_id" });
db.borrowRecords.createIndex({ record_id: 1 }, { unique: true, name: "uq_borrow_record_id" });

// Note: creating a unique index will fail if duplicates exist. We inserted unique ids above, so it's safe.
```

```
// ============ 4. Performance test with explain() ============
// Strategy:
// - Create a copy of the collection without indexes to act as "without index" baseline.
// - Run .explain("executionStats") on both collections for the same query and compare totalDocsExamined / executionTimeMillis.

// Example Query A: Find book by a specific author quickly.
// Query: Find books where authors array contains "A001" (R. K. Narayan)

// Create a copy for baseline (no indexes)
db.books.aggregate([{ $match: {} }, { $out: "books_copy_noindex" }]);

// Ensure books_copy_noindex has no indexes except _id
print("Indexes on books_copy_noindex:");
printjson(db.books_copy_noindex.getIndexes());

// A1: Explain on copy (no book text/index) - baseline
print("\n=== Explain (no index) : find books by author A001 ===");
let qA = { authors: "A001" };
printjson(db.books_copy_noindex.find(qA).explain("executionStats"));

// A2: Explain on real books collection (with index on book_id and text index).
// Note: authors array is not indexed - we can create an index for authors to show difference
db.books.createIndex({ authors: 1 }, { name: "idx_books_authors" });

print("\n=== Explain (with authors index) : find books by author A001 ===");
printjson(db.books.find(qA).explain("executionStats"));


// Example Query B: Search books by title keyword (text index)
// B1: explain on copy without text index (books_copy_noindex)
let qB = { $text: { $search: "Malgudi" } };
print("\n=== Explain (no index) : text search 'Malgudi' on books_copy_noindex ===");
printjson(db.books_copy_noindex.find(qB).explain("executionStats"));

// B2: explain on books with text index
print("\n=== Explain (with text index) : text search 'Malgudi' on books ===");
printjson(db.books.find(qB).explain("executionStats"));


// Example Query C: Find overdue books (dueDate < now and returned:false)
// For repeatability, set "now" variable
const now = new Date();
let qC = { dueDate: { $lt: now }, returned: false };
```

```
// Create copy baseline for borrowRecords
db.borrowRecords.aggregate([{ $match: {} }, { $out: "borrowRecords_copy_noindex" }]);

print("\nIndexes on borrowRecords_copy_noindex:");
printjson(db.borrowRecords_copy_noindex.getIndexes());

// C1: explain on copy (no index)
print("\n=== Explain (no index) : find overdue borrowRecords ===");
printjson(db.borrowRecords_copy_noindex.find(qC).explain("executionStats"));

// To speed up this query, create partial or compound indexes. We'll create a compound index (returned + dueDate)
db.borrowRecords.createIndex({ returned: 1, dueDate: 1 }, { name: "idx_borrow_returned_dueDate" });

print("\n=== Explain (with idx_borrow_returned_dueDate) : find overdue borrowRecords ===");
printjson(db.borrowRecords.find(qC).explain("executionStats"));


// Example Query D: Get borrowing history of a specific member sorted by date
let qD = { member_id: "M001" };

// D1: baseline (copy has no compound index)
db.borrowRecords_copy_noindex.dropIndexes(); // ensure only _id exists on copy for fairness
print("\n=== Explain (no index) : borrow history for M001 (no index) ===");
printjson(db.borrowRecords_copy_noindex.find(qD).sort({ borrowDate: -1 }).explain("executionStats"));

// D2: with compound index (member_id + borrowDate desc) on real collection
print("\n=== Explain (with idx_borrow_member_borrowDate) : borrow history for M001 ===");
printjson(db.borrowRecords.find(qD).sort({ borrowDate: -1 }).explain("executionStats"));


// ============ 5. Additional example queries to run interactively ============
/*
i) Find books by a specific author quickly:
  db.books.find({ authors: "A001" })

ii) Search books by title keyword (text index):
  db.books.find({ $text: { $search: "Malgudi" } }, { score: { $meta: "textScore" } }).sort({ score: { $meta: "textScore" } })

iii) Find overdue books efficiently:
  db.borrowRecords.find({ dueDate: { $lt: new Date() }, returned: false })

iv) Get borrowing history of a specific member, sorted by date:
```

```
  db.borrowRecords.find({ member_id: "M001" }).sort({ borrowDate: -1 })
*/

// ============ 6. Useful helper commands ============
print("\n--- Indexes summary ---");
print("books indexes:");
printjson(db.books.getIndexes());
print("borrowRecords indexes:");
printjson(db.borrowRecords.getIndexes());
print("members indexes:");
printjson(db.members.getIndexes());
print("authors indexes:");
printjson(db.authors.getIndexes());

print("\n--- Collection counts ---");
printjson({
  books: db.books.countDocuments(),
  books_copy_noindex: db.books_copy_noindex.countDocuments(),
  borrowRecords: db.borrowRecords.countDocuments(),
  borrowRecords_copy_noindex: db.borrowRecords_copy_noindex.countDocuments()
});

print("\n--- Demo complete ---");
```

6. Create a simple application that uses MongoDB with python.
Note: To install pymongo package: pip install pymongo

```
import tkinter as tk
from tkinter import messagebox
from pymongo import MongoClient

# MongoDB connection
client = MongoClient("mongodb://localhost:27017/")
db = client["collegeDB"]
collection = db["students"]

# Insert student record into MongoDB
def insert_student():
    name = name_entry.get()
    roll = roll_entry.get()
    branch = branch_entry.get()
    cgpa = cgpa_entry.get()

    if not (name and roll and branch and cgpa):
        messagebox.showwarning("Input Error", "All fields are required!")
```

```
        return

    try:
        cgpa = float(cgpa)
        collection.insert_one({"name": name, "roll": roll, "branch": branch, "cgpa": cgpa})
        messagebox.showinfo("Success", "Student record inserted.")
        name_entry.delete(0, tk.END)
        roll_entry.delete(0, tk.END)
        branch_entry.delete(0, tk.END)
        cgpa_entry.delete(0, tk.END)
    except ValueError:
        messagebox.showerror("Invalid Input", "CGPA must be a number.")

# Display all student records
def display_students():
    students = collection.find()
    result_text.delete("1.0", tk.END)
    for s in students:
        result_text.insert(tk.END, f"{s['name']} | {s['roll']} | {s['branch']} | {s['cgpa']}\n")

# GUI setup
app = tk.Tk()
app.title("Student Database App (MongoDB)")

tk.Label(app, text="Name:").grid(row=0, column=0)
name_entry = tk.Entry(app)
name_entry.grid(row=0, column=1)

tk.Label(app, text="Roll No:").grid(row=1, column=0)
roll_entry = tk.Entry(app)
roll_entry.grid(row=1, column=1)

tk.Label(app, text="Branch:").grid(row=2, column=0)
branch_entry = tk.Entry(app)
branch_entry.grid(row=2, column=1)

tk.Label(app, text="CGPA:").grid(row=3, column=0)
cgpa_entry = tk.Entry(app)
cgpa_entry.grid(row=3, column=1)

tk.Button(app, text="Insert Student", command=insert_student).grid(row=4, column=0,
columnspan=2, pady=10)
tk.Button(app, text="Show All Students", command=display_students).grid(row=5,
column=0, columnspan=2, pady=5)

result_text = tk.Text(app, width=50, height=10)
result_text.grid(row=6, column=0, columnspan=2, padx=10, pady=10)
```

```
app.mainloop()
```

7. **Demonstrate how sharding improves query performance in MongoDB by comparing query execution on an unsharded collection vs a sharded collection.**
   Pre-requisites

   - **MongoDB sharded cluster is already set up (at least 2 shards).**
   - **Student is connected to mongos router via mongo.**

Before Sharding:

```
from pymongo import MongoClient, ASCENDING
from pprint import pprint
import time

client = MongoClient("mongodb://127.0.0.1:27017")
db = client["demo_db"]
col = db["orders"]

def robust_explain(cursor):
    """
    Accepts a Cursor (returned by col.find(query).limit(n)) and returns an explain dict.
    Tries the common forms across PyMongo versions:
      - cursor.explain(verbosity="executionStats")   (PyMongo 4.x)
      - cursor.explain("executionStats")             (older PyMongo)
      - cursor.explain()                             (some servers/clients default)
    Falls back to db.command("explain", {...}) if cursor.explain fails.
    """
    # 1) Try PyMongo 4.x style
    try:
        return cursor.explain(verbosity="executionStats")
    except TypeError:
        pass
    except Exception:
        # unexpected but keep trying fallback options
        pass

    # 2) Try older positional style
    try:
        return cursor.explain("executionStats")
    except TypeError:
        pass
    except Exception:
        pass

    # 3) Try no-arg style
```

```
    try:
        return cursor.explain()
    except TypeError:
        pass
    except Exception:
        pass

    # 4) Fallback: use the server-side explain command directly.
    # Build a minimal command for a find explain.
    try:
        # cursor._Cursor__query may be implementation-dependent; build explicitly instead
        # We fetch the query and projection/limit/skip from the cursor where possible.
        # For simplicity, require the caller to pass query, projection and limit when using fallback.
        raise RuntimeError("Cursor.explain variants failed - use db.command fallback.")
    except RuntimeError:
        raise

def explain_by_command(collection, query, projection=None, limit=100):
    """
    DB command fallback that works across server versions.
    Returns the explain by running the 'explain' command for a find.
    """
    cmd = {"explain": {"find": collection.name, "filter": query, "limit": limit}}
    if projection:
        cmd["explain"]["projection"] = projection
    # request verbosity
    cmd["verbosity"] = "executionStats"
    return collection.database.command(cmd)

# Example unify wrapper to use in your script
def explain_and_time(collection, query, limit=100):
    # try cursor-based explain first
    cursor = collection.find(query).limit(limit)
    try:
        t0 = time.time()
        ex = robust_explain(cursor)
        t1 = time.time()
        # If robust_explain raised RuntimeError then it'll be caught below
    except RuntimeError:
        # fallback via db.command
        t0 = time.time()
        ex = explain_by_command(collection, query, limit=limit)
        t1 = time.time()
```

```
    result = {
        "wall_ms": int((t1 - t0) * 1000),
        "nReturned": ex.get("executionStats", {}).get("nReturned"),
        "totalDocsExamined": ex.get("executionStats", {}).get("totalDocsExamined"),
        "totalKeysExamined": ex.get("executionStats", {}).get("totalKeysExamined"),
        "executionTimeMillis": ex.get("executionStats", {}).get("executionTimeMillis"),
    }
    return ex, result

# -------------------------
# Demo usage:
# -------------------------
# seed some data if empty (only run once)
if col.count_documents({}) == 0:
    docs = []
    for i in range(1, 2001):
        docs.append({
            "orderId": f"O{i}",
            "customerId": "C" + str(1 + (i % 500)),
            "amount": (i % 200) + 0.5,
            "status": "completed" if i % 5 else "cancelled"
        })
    col.insert_many(docs)
    print("Inserted sample documents")

col.create_index([("customerId", ASCENDING)], name="idx_customer")

q_targeted = {"customerId": "C100"}
q_nonkey = {"status": "completed", "amount": {"$gt": 50}}

ex, summary = explain_and_time(col, q_targeted)
print("--- Targeted query ---")
pprint(summary)
# optional: inspect full explain
# pprint(ex)

ex, summary = explain_and_time(col, q_nonkey)
print("--- Non-key query ---")
pprint(summary)
```

After Sharding:

```
from pymongo import MongoClient, ASCENDING
from pprint import pprint
import time
```

```
# Connect to mongos (replace host/port if needed)
client = MongoClient("mongodb://127.0.0.1:27017")
db = client["demo_db"]
col = db["orders"]

def explain_on_mongos(collection, query, projection=None, limit=100, verbosity="executionStats"):
    """
    Run an explain via the DB command so that mongos will return per-shard explain info.
    Returns a tuple: (explain_doc, summary_dict)
    summary_dict contains:
      - top-level execution stats (if present)
      - shards: dict keyed by shard name with per-shard stats
      - aggregates: sum of docs examined / keys examined across shards (if shard info present)
    """
    cmd = {
        "explain": {
            "find": collection.name,
            "filter": query,
            "limit": limit
        },
        "verbosity": verbosity
    }
    if projection:
        cmd["explain"]["projection"] = projection

    ex = collection.database.command(cmd)  # send to mongos; returns explain doc
    summary = {}

    # Top-level stats (if present)
    exec_stats = ex.get("executionStats", {})
    if exec_stats:
        summary["nReturned"] = exec_stats.get("nReturned")
        summary["totalDocsExamined"] = exec_stats.get("totalDocsExamined")
        summary["totalKeysExamined"] = exec_stats.get("totalKeysExamined")
        summary["executionTimeMillis"] = exec_stats.get("executionTimeMillis")

    # Per-shard breakdown (present when explain is run through mongos on a sharded cluster)
    shards = ex.get("shards")
    if shards and isinstance(shards, dict):
        summary["shards"] = {}
        total_docs = 0
        total_keys = 0
```

```
    total_exec_ms = 0
    for shard_name, shard_ex in shards.items():
        s_exec = shard_ex.get("executionStats", {})
        s_nReturned = s_exec.get("nReturned", 0)
        s_docsExamined = s_exec.get("totalDocsExamined", 0)
        s_keysExamined = s_exec.get("totalKeysExamined", 0)
        s_execMs = s_exec.get("executionTimeMillis", 0)

        summary["shards"][shard_name] = {
            "nReturned": s_nReturned,
            "totalDocsExamined": s_docsExamined,
            "totalKeysExamined": s_keysExamined,
            "executionTimeMillis": s_execMs,
            # also keep the winningPlan stage if available (quick hint)
            "winningPlanStage":      shard_ex.get("queryPlanner",      {}).get("winningPlan",
{}).get("stage")
        }

        total_docs += s_docsExamined
        total_keys += s_keysExamined
        # executionTimeMillis is per-shard; aggregate is approximate (sum)
        total_exec_ms += s_execMs

    summary["aggregate"] = {
        "totalDocsExamined_across_shards": total_docs,
        "totalKeysExamined_across_shards": total_keys,
        "sumExecutionTimeMillis_shards": total_exec_ms
    }

  return ex, summary

# -------------------------
# Example usage & printing
# -------------------------
# ensure index exists for demonstration
col.create_index([("customerId", ASCENDING)], name="idx_customer")

q_targeted = {"customerId": "C100"}
q_nonkey = {"status": "completed", "amount": {"$gt": 50}}

# Targeted query explain (should be routed to 1 shard when sharded on customerId)
ex_doc, summary = explain_on_mongos(col, q_targeted, limit=100)
print("\n== Targeted Query Explain Summary ==")
pprint(summary)
```

```
# if you want to inspect the raw explain (very verbose), uncomment:
# pprint(ex_doc)

# Non-shard-key query (scatter-gather across shards)
ex_doc2, summary2 = explain_on_mongos(col, q_nonkey, limit=100)
print("\n== Non-Shard-Key Query Explain Summary ==")
pprint(summary2)
```